CS69021: Computing Lab-1

Practise Assignment || August 05, 2026

Topic: Dynamic Programming & Recursion

========== Instructions ==========

- Read the problem. Type it yourself, from scratch, without any AI help. Get it wrong. Debug it. That struggle is literally the only part of this that builds the skill you're being tested on.
- For DP problems, first try the recursive solution, then add memoization, and finally convert it to tabulation. Don't directly jump to the optimized solution.
- Always check the constraints before coding. Use them to figure out what time and space complexity will actually pass.
- Learn to handle integer overflow properly. Know when to use long long, where intermediate calculations can overflow, and how to handle modulo operations correctly.
- Submit whatever you have done by 5 PM on Moodle in a single zip <rollno> wk prc.zip. Do the rest at home.

==========

Q1) Subset Sum Count

Problem Statement: Given an array arr of *n* integers and an integer *K*, count the number of subsets of the given array that have a sum equal to *K*.

Reference: https://www.geeksforgeeks.org/problems/perfect-sum-problem5633/1

Input format:

Line 1: integer *n* (size of the array)

Line 2: *n* space-separated integers (the array elements)

Line 3: integer *K* (the target sum)

Output format: A single integer - the number of subsets of arr whose elements sum to *K* (two subsets that pick different indices are counted separately, even if the values are equal).

Test	Input	Output	Explanation
1	4 1 2 3 3 6	3	The subsets (by index) that sum to 6 are {1,2,3} using the first 3, {1,2,3} using the second 3, and {3,3} - 3 subsets in total.
2	5 1 1 1 1 1 3	10	Any 3 of the five 1's sum to 3, and there are $\binom{5}{3}$ = 10 ways to choose 3 indices out of 5.

Q2) Pow(x, n)

Problem Statement: Implement pow(x, n), i.e. compute x raised to the power n (n can be negative or zero). First write a straightforward recursive solution, and once that works, rewrite it so it runs in $O(\log n)$ time.

LeetCode 50: https://leetcode.com/problems/powx-n/

Input format: A single line containing a real number x and an integer n, space-separated.

Output format: The value of x^n, printed correct to 5 decimal places.

Test	Input	Output	Explanation
1	2.00000 10	1024.00000	$2^{10} = 1024$.
2	2.00000 -2	0.25000	A negative exponent means $x^n = 1/x^{\lvert n\rvert}$, so $2^{-2} = 1/4 = 0.25$.

Q3) Longest Palindromic Subsequence

Problem Statement: Given a string s, find the length of the longest subsequence of s that is a palindrome. A subsequence need not use consecutive characters, but must preserve their relative order.

LeetCode 516: https://leetcode.com/problems/longest-palindromic-subsequence/ **Input format:** A single line - the string s.

Output format: A single integer - the length of the longest palindromic subsequence.

Test	Input	Output	Explanation
1	bbbab	4	"bbbb" (drop the middle a) is a palindromic subsequence of length 4, and no length-5 subsequence of bbbab is a palindrome.
2	cbbd	2	"bb" is the longest palindromic subsequence; c and d can't be matched with anything to extend it.

Q4) House Robber II

Problem Statement: You are a professional robber planning to rob houses along a street, where the houses are arranged in a circle (so the first house and the last house are adjacent). Each house i holds nums[i] amount of money. You cannot rob two adjacent houses on the same night, or the police get alerted. Find the maximum amount of money you can rob without alerting the police. *LeetCode 213:* https://leetcode.com/problems/house-robber-ii/

Input format:

Line 1: integer n

Line 2: n space-separated integers (nums, arranged in a circle)

Output format: A single integer - the maximum amount of money that can be robbed.

Test	Input	Output	Explanation
1	3 2 3 2	3	Because it's a circle, house 1 and house 3 are adjacent too, so you can't take both. The best single pick is house 2, worth 3.
Test	**Input**	**Output**	**Explanation**
2	4 1 2 3 1	4	Rob house 1 (value 1) and house 3 (value 3); they aren't adjacent on the circle, giving a total of 4, which beats any other combination.

Q5) Backtracking: N-Queens & Word Search Q5a) N-Queens

Problem Statement: The n-queens puzzle asks you to place n queens on an $n \times n$ chessboard such that no two queens attack each other (no two share a row, a column, or a diagonal). Given n, count the total number of distinct solutions.

LeetCode 51 (N-Queens): https://leetcode.com/problems/n-queens/
LeetCode 52 (N-Queens II - count only): https://leetcode.com/problems/n-queens-ii/

Input format: A single integer n.

Output format: A single integer - the number of distinct solutions to the n-queens puzzle.

Test	Input	Output	Explanation
1	4	2	For a 4 × 4 board there are exactly two arrangements of 4 non-attacking queens (they are mirror images of each other).
2	1	1	A single queen on a 1 × 1 board trivially doesn't attack anything, so there's exactly one solution.

Q5b) Word Search

Problem Statement: Given an $m \times n$ grid of characters board and a string word, determine if word exists in the grid. The word must be built from letters of sequentially adjacent cells, where "adjacent" cells are horizontally or vertically neighbouring (no diagonals), and the same cell may not be reused within one word. Keep this one optimal: prune before you branch - e.g. check letterfrequency feasibility up front, and cut a DFS branch the moment it can't possibly complete the word, instead of exploring every path blindly.

LeetCode 79: https://leetcode.com/problems/word-search/

Input format:

Line 1: two integers m n (rows and columns of the board)

Next m lines: a string of n characters each (one row of the board)

Last line: the string word to search for

Output format: Print true if word exists in the grid, else print false.

Test	Input	Output	Explanation
1	3 4 ABCE SFCS ADEE ABCCED	true	Starting at the top-left A, the path A(0,0)→B(0,1)→C(0,2)→C(1,2)→ spells “ABCCED” using only up/down/left/right moves without reusing a cell.
2	3 4 ABCE SFCS ADEE ABCB	false	After A→B→C, the only unused neighbour of C that equals ‘B’ doesn’t exist adjacent to it without revisiting a used cell, so “ABCB” cannot be traced out.

E(2,2)→ D(2,1)